Sprint 3 | Parte 1 - Suíte LLM e análise crítica

Projeto P07 – Controle de Frequência · Base: c9028b6, feature de token temporário · **Relatório gerado a partir dos logs locais**.

1. Contexto

Quatro user stories selecionadas: US01 abertura de chamada temporária; US02 registro seguro por aluno; US03 consulta de janelas; US04 frequência e nível de faltas. Backend NestJS/TypeScript, Prisma/PostgreSQL, Vitest 4.x e @vitest/coverage-v8 alinhado à mesma versão. LLM: GPT-6, OpenAI (09/11/2026); detalhes e ressalvas de independência em llm-log.md e llm-log-v2.md. Os testes usam mocks do Prisma: **não são testes de concorrência real no PostgreSQL**.

2. Projeto manual — antes da LLM

O projeto preparatório matriz-projeto-manual.md define **15 critérios**, **30 partições** válidas/inválidas e **15 fronteiras**, em quatro US. A matriz preparatória foi elaborada com auxílio de LLM e registrada antes da geração da V1, conforme o histórico do repositório. A ratificação humana dos critérios deve ser confirmada pela equipe. A matriz final mapeia CT01–CT24 e EVO01–EVO15. Exemplos: duração 0/1/15/16 min, tentativas 5/6, código com 7/8/9 caracteres e faltas 14,9/15/25/25,1%.

3. Geração — protocolo LLM

A v1 contém 24 testes em tests/v1-llm/, com comentários sobre requisitos e técnica. O log informa o modelo, contexto, prompt reproduzível e limitações da reconstrução; a v1 é preservada após seu commit. A v2 preserva os mesmos 24 testes em tests/v2/base/ e adiciona **15 testes EVO**. As duas configurações executam suites independentes, no mesmo conjunto de cinco arquivos de produção. Não foram reaproveitados testes ou medições da Sprint 3 anterior. A existência de reparos para compilação deve ser conferida no histórico do Git, sem editar a v1 silenciosamente.

4. Indicadores — comparação real

Indicador	V1 gerada por LLM	V2 melhorada
Gerados / passando	24 / 24	39 / 39
Falhando / não compilando	0 / 0	0 / 0
Cobertura de linhas	66,30%	92,39%
Cobertura de branches	64,81%	85,18%
Cobertura dos 15 critérios	12/15 = 80,0%	15/15 = 100,0%
Partições cobertas (30)	20/30 = 66,7%	30/30 = 100,0%
Valores-limite (15)	10/15 = 66,7%	15/15 = 100,0%
Sem asserção explícita	0	0
Defeitos reais comprovados	0 nas execuções	0 nas execuções

Evidência: docs/testes/relatorios/reconstrucao/{v1,v2}/coverage-summary.json e index.html; logs execucoes/{v1,v2,baseline,build}.txt; indicadores detalhados em indicadores-medidos.md. Ambos têm o MESMO escopo (5 arquivos), incluindo frequencia.service.ts nos denominadores da v1 e v2. A execução baseline da aplicação teve 58 testes em 10 arquivos.

5. Análise crítica — quatro perguntas da Etapa 4

P1 – Partições/limites perdidos, inventados e extras. A v1 cobriu, segundo a matriz revisada, 20 de 30 partições e 10 de 15 limites. Faltaram os estados P21–P30, particularmente tentativas 5/6, presença preexistente, status na fronteira e consultas de frequência; faltaram L05/L07 (código 7/9), L13 (expiração exata) e L14/L15 (limite de tentativas). A v1 acrescentou uma restrição **não requerida explicitamente**: em CT09, codigoHash é verificado com regex de 64 caracteres hexadecimais, pressupondo SHA-256 em vez de apenas “hash não reversível”. Não identificamos outro caso funcional comprovado que a LLM acertou fora do gabarito manual; não inventamos métrica de novidade. A v2 contempla as ausências conforme a matriz final.

P2 – Critérios omitidos ou testes sem requisito. A v1 não cobre integralmente US02-CA04 (limite de tentativas), US03-CA03 (status e contagem) e US04-CA03 (detalhe/minhas), resultando em 12/15 critérios. A v2 os inclui com EVO01–02, EVO07–08 e EVO10–13, atingindo 15/15 pela rastreabilidade. CT09 (tamanho do hash) é uma asserção de implementação sem critério específico sobre algoritmo, portanto sua pertinência precisa ser reavaliada pelo grupo.

P3 – Linhas versus branches. Linha executada não garante os dois resultados de uma decisão. Trecho real de chamada-codigo.service.ts (commit-base):

```
if (tentativa.total > MAX_TENTATIVAS) return 'bloqueada' as const;
if (!codigoConfere(codigo.trim().toUpperCase(), atual.codigoHash, atual.salt)) {
```

```
  return 'invalida' as const;
}
```

A v1 exercita o bloco com tentativa 1, portanto há linha percorrida mas **não** a ramificação total > 5, que só EVO02 cobre. Os valores medidos de branches são 64,81% e 85,18% no escopo fixo. Branches atribuídos pelo V8 a @Injectable() podem ser instrumentação sintética, não if de negócio; confira coverage-final.json antes de interpretá-los.

P4 – Oráculos e falsos positivos. Inspeção dos 24 testes v1 e 39 testes v2 identificou **0 casos sem chamada explícita a expect**. Há **1 oráculo claramente copiado do detalhe da implementação**: a exigência de SHA-256 hexadecimal com 64 caracteres em CT09. Existem ainda asserções de forma de consultas/mock Prisma cuja adequação deve ser checada contra o contrato desejado, mas não as contamos como defeito provado. **0 testes passando e validando defeito real foram demonstrados**; isso não equivale a inexistência de bugs.

6. Melhorias específicas da v2

Mudança	Lacuna da v1 resolvida	Evidência do teste
EVO01–02	quinta tentativa ainda válida; sexta bloqueada	ramo > MAX_TENTATIVAS
EVO03–04	expiração exata e encerramento manual	condição de estaAtiva
EVO05–06	presença idempotente e normalização no serviço	transação e hash de código
EVO07–09	status, contagem e presença na listagem	caminhos de consulta
EVO10–14	FrequenciaService sem cobertura v1	5 cenários de detalhe, minhas e zero aulas
EVO15	fronteiras de comprimento 7/8/9 letras	schema de registro

O escopo de cobertura permanece fixo; portanto, o aumento estrutural não pode ser explicado por mudança de denominador. Os indicadores de requisitos são **classificação manual de rastreabilidade** e os percentuais estruturais são medidos pelo V8. Não confundir os dois.

7. Conclusão — quando voltar a usar LLM para testes

A equipe pode usar LLM para levantar hipóteses de partições e acelerar testes repetitivos com mocks; não deve usá-la como único oráculo. Procedimento recomendado: definir critérios antes, preservar a primeira saída, investigar erros, exigir testes específicos para ramos e limites faltantes, verificar asserções contra requisitos, inspecionar cobertura e manter testes de integração para a concorrência/transações reais. Uma suíte com 100% de linhas não certifica correção nem detecta tudo que o sistema deveria fazer.

Apêndices (no repositório): matriz-projeto-manual.md, matriz-rastreabilidade.md, llm-log.md, llm-log-v2.md, indicadores-medidos.md, JSON/HTML e logs. Os indicadores estruturais foram extraídos dos relatórios de cobertura, enquanto a cobertura de requisitos foi classificada pela matriz de rastreabilidade.